TALLER PRÁCTICO

Taller 2: Regresión Lineal y Logística con Datos Reales

Predicción de Resultados Premier League | Machine Learning I (ML1-2026I)

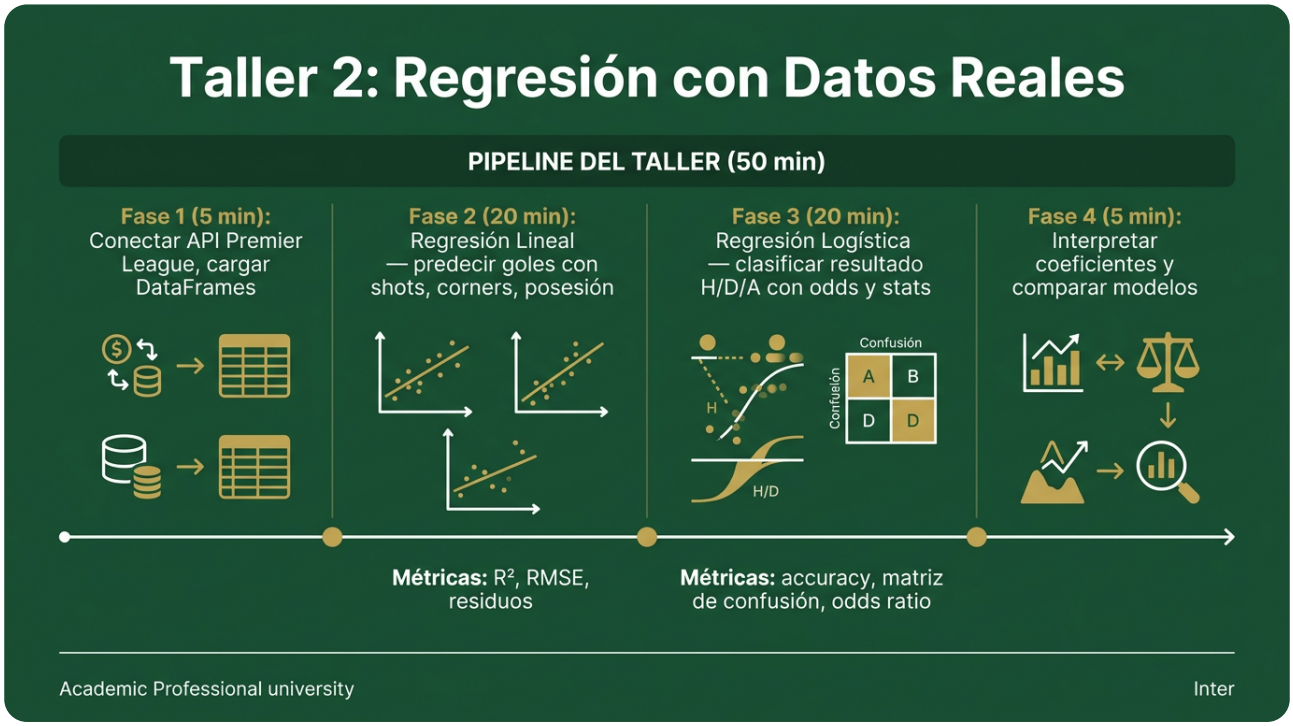
Julián Zuluaga — Docente

16 de marzo de 2026

Taller 2: Regresión Lineal y Logística con Datos Reales

Predicción de Resultados en la Premier League 2025-26

Asignatura: Machine Learning I (ML1-2026I) **Fecha:** 16 de marzo de 2026 **Duración:** 50 minutos (guiado en clase) **Peso:** 15% de la nota final **Entrega individual:** Notebook `.ipynb` por correo — **17 de abril de 2026**



Estructura del taller: 4 fases en 50 minutos

Objetivo

Aplicar regresión lineal y logística a datos reales de la Premier League 2025-26, consumidos directamente desde una API REST. El taller cubre el pipeline completo: **conexión → exploración → modelado → interpretación**.

Al finalizar, el estudiante será capaz de:

1. Consumir una API REST con `requests` y construir DataFrames
2. Entrenar un modelo de regresión lineal y evaluar con R^2, RMSE
3. Entrenar un modelo de regresión logística y evaluar con accuracy, matriz de confusión
4. Interpretar coeficientes y odds ratios en contexto deportivo

---

Recursos

Recurso	URL
API Premier League	`https://premier.72-60-245-2.sslip.io`
Documentación interactiva	`https://premier.72-60-245-2.sslip.io/docs`
Docs para LLMs	`https://premier.72-60-245-2.sslip.io/llms.txt`

> **Nota:** La API no requiere autenticación. Los datos se actualizan diariamente.

Fase 1 — Conexión a la API (5 min)

1.1 Instalar dependencias

```
# En Google Colab, estas librerías ya están disponibles
import requests
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (mean_squared_error, r2_score,
                             accuracy_score, confusion_matrix,
                             classification_report)
```

1.2 Conectar a la API

```
BASE_URL = "https://premier.72-60-245-2.sslip.io"

# Verificar conexión
health = requests.get(f"{BASE_URL}/health").json()
print(f"API Status: {health['status']}")
print(f"Partidos: {health['data_counts']['matches']}")
print(f"Jugadores: {health['data_counts']['players']}")
```

1.3 Cargar datos de partidos

```
# Cargar TODOS los partidos (291 jugados hasta GW29)
response = requests.get(f"{BASE_URL}/matches?limit=500")
matches = pd.DataFrame(response.json()["matches"])
print(f"Partidos cargados: {len(matches)}")
matches.head()
```

1.4 Exploración rápida

```
# Columnas disponibles
print("Columnas:", matches.columns.tolist())

# Resultado más común (ftr = Full Time Result: H/D/A)
print("\nDistribución de resultados:")
print(matches["ftr"].value_counts())

# Promedio de goles (fthg = Full Time Home Goals, ftag = Away Goals)
print(f"\nGoles por partido: {(matches['fthg'] + matches['ftag']).mean():.2f}")
```

Datos disponibles en la API: 822 jugadores, 291 partidos, 444K eventos

Fase 2 — Regresión Lineal: Predecir Goles (20 min)

Pregunta: ¿Podemos predecir cuántos goles marca el equipo local a partir de sus estadísticas del partido?

2.1 Preparar features

```
# Variable objetivo: goles del equipo local (fthg = Full Time Home Goals)
y = matches["fthg"].astype(float)

# Features numéricas del equipo local
# hs=shots, hst=shots on target, hc=corners, hf=fouls, hy=yellow, hr=red
features = ["hs", "hst", "hc", "hf", "hy", "hr"]

X = matches[features].astype(float)

# Verificar que no hay NaNs
print(f"NaN en X: {X.isna().sum().sum()}")
print(f"Shape: X={X.shape}, y={y.shape}")
```

2.2 Dividir train/test

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Train: {len(X_train)} | Test: {len(X_test)}")
```

2.3 Entrenar modelo

```
model_lr = LinearRegression()
model_lr.fit(X_train, y_train)

# Predicciones
y_pred = model_lr.predict(X_test)

# Métricas
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"R² = {r2:.3f}")
print(f"RMSE = {rmse:.3f}")
```

2.4 Interpretar coeficientes

```
coefs = pd.DataFrame({
    "Feature": features,
    "Coeficiente": model_lr.coef_
}).sort_values("Coeficiente", ascending=False)

print(coefs.to_string(index=False))
print(f"\nIntercepto: {model_lr.intercept_:.3f}")
```

> **Pregunta para reflexionar:** ¿Cuál feature tiene mayor impacto? ¿Tiene sentido que `hst` (shots on target) sea la más importante?

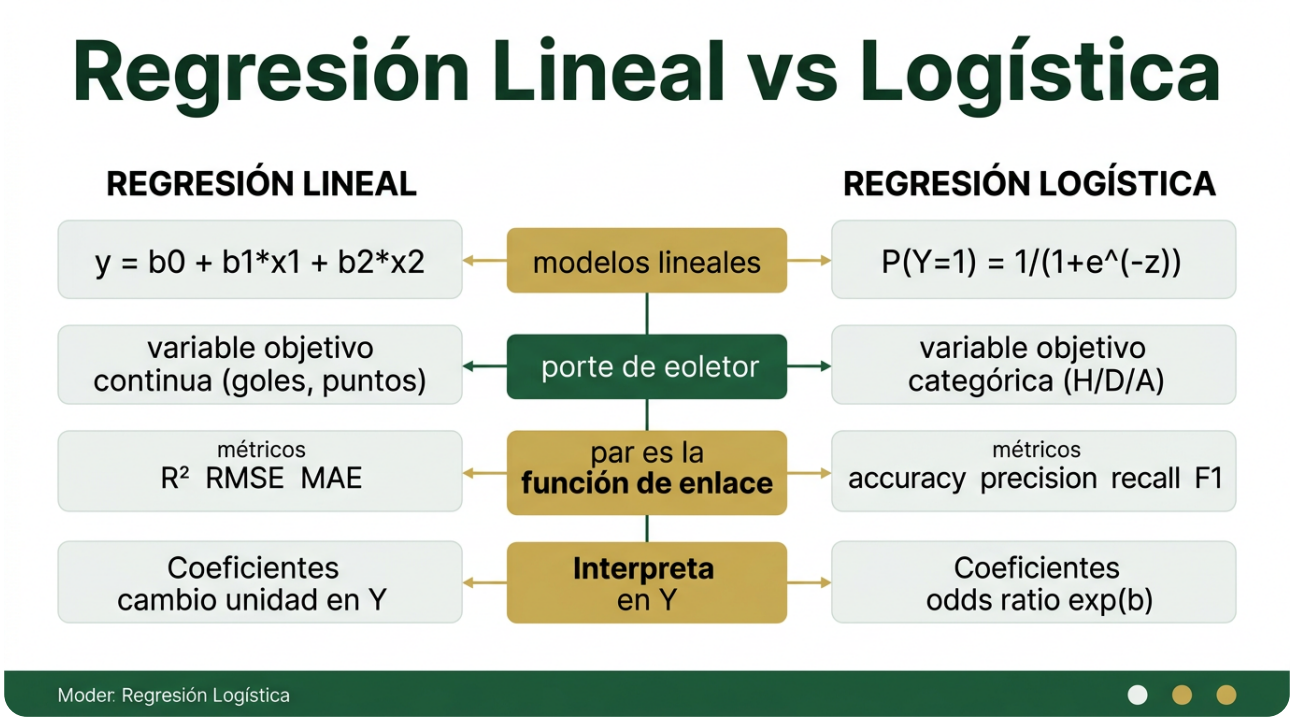
2.5 Visualizar predicciones vs realidad

```
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, alpha=0.5, edgecolors='k')
plt.plot([0, 6], [0, 6], 'r--', label='Predicción perfecta')
plt.xlabel("Goles reales")
plt.ylabel("Goles predichos")
plt.title(f"Regresión Lineal — R² = {r2:.3f}")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

2.6 Análisis de residuos

```
residuos = y_test - y_pred

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Residuos vs predichos
axes[0].scatter(y_pred, residuos, alpha=0.5, edgecolors='k')
axes[0].axhline(y=0, color='r', linestyle='--')
axes[0].set_xlabel("Valores predichos")
axes[0].set_ylabel("Residuos")
axes[0].set_title("Residuos vs Predichos")

# Histograma de residuos
axes[1].hist(residuos, bins=15, edgecolor='k', alpha=0.7)
axes[1].set_xlabel("Residuo")
axes[1].set_ylabel("Frecuencia")
axes[1].set_title("Distribución de Residuos")

plt.tight_layout()
plt.show()

print(f"Media residuos: {residuos.mean():.4f} (debe ser ≈ 0)")
print(f"Std residuos: {residuos.std():.4f}")
```

> **Análisis esperado:** Los residuos deben estar centrados en 0 y distribuidos simétricamente. Si no lo están, el modelo tiene sesgo sistemático.



Dos modelos lineales, dos funciones de enlace — la diferencia clave

Fase 3 — Regresión Logística: Clasificar Resultado (20 min)

Pregunta: ¿Podemos predecir si gana el local (H), empate (D) o gana el visitante (A)?

3.1 Preparar target y features

```
# Target: resultado del partido (H, D, A) — ftr = Full Time Result
y_class = matches["ftr"]
print(f"Distribución:\n{y_class.value_counts()}")
print(f"\nBalance: {y_class.value_counts(normalize=True).round(3)}")
```

3.2 Features de apuestas (probabilidades implícitas)

```
# Las odds de Bet365 son probabilidades del mercado
# Convertir a probabilidades implícitas: prob = 1/odd
odds_features = ["b365h", "b365d", "b365a"]
X_odds = matches[odds_features].astype(float)

# Convertir odds a probabilidades
X_probs = 1 / X_odds
X_probs.columns = ["prob_H", "prob_D", "prob_A"]

# Normalizar (las probs suman >1 por el margen de la casa)
row_sums = X_probs.sum(axis=1)
X_probs = X_probs.div(row_sums, axis=0)

print(X_probs.head())
print(f"\nSuma de probabilidades (debe ser 1.0): {X_probs.sum(axis=1).mean():.4f}")
```

3.3 Combinar features

```
# Combinar probabilidades + stats del partido
# hs/as_ = home/away shots, hst/ast = home/away shots on target
stats_features = ["hs", "as_", "hst", "ast"]
X_combined = pd.concat([X_probs, matches[stats_features].astype(float)], axis=1)

# Eliminar filas con NaN
mask = ~(X_combined.isna().any(axis=1) | y_class.isna())
X_clean = X_combined[mask]
y_clean = y_class[mask]

print(f"Registros limpios: {len(X_clean)}")
```

3.4 Entrenar modelo

```
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clean, y_clean, test_size=0.2, random_state=42, stratify=y_clean
)

model_log = LogisticRegression(
    multi_class='multinomial',  # 3 clases: H, D, A
    max_iter=1000,
    random_state=42
)
model_log.fit(X_train_c, y_train_c)

# Predicciones
y_pred_c = model_log.predict(X_test_c)
acc = accuracy_score(y_test_c, y_pred_c)
print(f"Accuracy: {acc:.3f}")
```

3.5 Matriz de confusión

```
from sklearn.metrics import ConfusionMatrixDisplay

fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(
    y_test_c, y_pred_c, ax=ax,
    display_labels=["Away", "Draw", "Home"],
    cmap="Greens"
)
ax.set_title(f"Matriz de Confusión — Accuracy: {acc:.1%}")
plt.tight_layout()
plt.show()
```

3.6 Reporte de clasificación

```
print(classification_report(y_test_c, y_pred_c))
```

> **Pregunta:** ¿Qué clase es la más difícil de predecir? ¿Por qué los empates son casi imposibles de clasificar correctamente?

3.7 Comparar con el baseline de Bet365

```
# Bet365 predice el resultado más probable (menor odd)
baseline_pred = X_probs.loc[X_test_c.index].idxmax(axis=1)
baseline_pred = baseline_pred.map({"prob_H": "H", "prob_D": "D", "prob_A": "A"})

baseline_acc = accuracy_score(y_test_c, baseline_pred)
print(f"Accuracy Bet365 (baseline): {baseline_acc:.3f}")
print(f"Accuracy modelo logístico:  {acc:.3f}")
print(f"Diferencia: {(acc - baseline_acc)*100:+.1f} puntos porcentuales")
```

3.8 Interpretar coeficientes (Odds Ratio)

```
# Coeficientes del modelo logístico
for i, clase in enumerate(model_log.classes_):
    print(f"\n--- Clase: {clase} ---")
    coefs_log = pd.DataFrame({
        "Feature": X_clean.columns,
        "Coeficiente": model_log.coef_[i],
        "Odds Ratio": np.exp(model_log.coef_[i])
    }).sort_values("Coeficiente", ascending=False)
    print(coefs_log.to_string(index=False))
```

> **Cómo leer el Odds Ratio:**
>
> - OR = 1.5 → cada unidad adicional en esa feature **multiplica por 1.5** la probabilidad de ese resultado
> - OR = 0.7 → cada unidad adicional **reduce un 30%** la probabilidad
> - OR = 1.0 → no tiene efecto

Fase 4 — Reflexión y Conclusiones (5 min)

4.1 Comparación de modelos

```
print("=" * 50)
print("RESUMEN DEL TALLER")
print("=" * 50)
print(f"\nRegresión Lineal (predecir goles):")
print(f"  R² = {r2:.3f}")
print(f"  RMSE = {rmse:.3f}")
print(f"\nRegresión Logística (clasificar resultado):")
print(f"  Accuracy = {acc:.1%}")
print(f"  Baseline Bet365 = {baseline_acc:.1%}")
```

4.2 Preguntas de cierre

Responde estas preguntas en tu notebook antes de entregar:

1. **¿Qué feature es más predictiva para goles del local?** Justifica con los coeficientes de la regresión lineal.
2. **¿Por qué los empates son difíciles de predecir?** Analiza la matriz de confusión y la distribución de clases.
3. **¿El modelo logístico supera a Bet365?** Si no, ¿qué features adicionales podrían ayudar?
4. **¿Qué supuestos de la regresión lineal se cumplen?** Usa el análisis de residuos como evidencia.
5. **¿Cómo mejorarías el modelo?** Propone al menos 2 ideas concretas (features derivadas, regularización, más datos, etc.)

Rúbrica de Evaluación

Criterio	Peso	5.0	3.0	1.0
Conexión API + EDA	15%	Carga datos, explora distribuciones, identifica patrones	Carga datos, exploración básica	No logra conectar o explorar
Regresión Lineal	25%	Modelo entrenado, R^2/RMSE		Solo entrena sin interpretar

Criterio	Peso	5.0	3.0	1.0
		calculados, residuos analizados, coeficientes interpretados	Modelo entrenado con métricas	
Regresión Logística	25%	Modelo multinomial, matriz confusión, reporte, comparación con baseline	Modelo con accuracy	Solo entrena sin evaluar
Interpretación	20%	Odds ratios explicados en contexto, reflexión sobre supuestos y limitaciones	Menciona coeficientes sin profundizar	Sin interpretación
Código y presentación	15%	Notebook limpio, comentado, reproducible, con markdown explicativo	Código funcional pero desordenado	Código con errores

---

Para Explorar Después (Opcional)

Estos endpoints de la API pueden enriquecer tu análisis:

```
# Cargar datos de jugadores
players = pd.DataFrame(
    requests.get(f"{BASE_URL}/players?limit=900").json()["players"]
)

# Top goleadores
top_scorers = players.nlargest(10, "goals_scored")[
    ["web_name", "team_name", "goals_scored", "expected_goals", "minutes"]
]
print(top_scorers)

# Eventos de un partido (datos de coordenadas x,y)
events = requests.get(f"{BASE_URL}/matches/1/events").json()
print(f"Eventos en el partido: {events['total_events']}")
```

Ideas para mejorar el modelo en la entrega:

- Agregar features derivadas: `sot_diff = hst - ast`
- Usar `shots_ratio = hs / (hs + as_)`
- Incorporar datos de jugadores (xG total del equipo)
- Probar regularización (Ridge, Lasso)
- Usar validación cruzada en lugar de un solo split

Machine Learning I (ML1-2026I) | Universidad Externado de Colombia Docente: Julián Zuluaga | Taller 2 — Regresión con Datos de la Premier League